見本: LINEトーク履歴の自動転記（架空データ）

A〜D列は既存シートの並びのまま、E列から右が足した列です。白い行は受信、黄色の行は担当者がメニューの送信画面から送った分です。データはすべて架空で、ローカルの模擬環境で確認した結果を表の形にしています。実際のGoogle・LINEでは確認していません。

トーク履歴

	A	B	C	D	E	F	G	H	I
1	受信時刻	ユーザー名	受信メッセージ	画像URL	向き	種類	相手ID	メッセージID	イベントID
2	2026/10/28 10:00:00	架空顧客A	見積もりの件で質問があります		受信	テキスト	Udemo0001	m001	EV001
3	2026/10/28 10:01:00	架空顧客A		https://drive.google.com/file/d/file1/view	受信	画像	Udemo0001	m002	EV002
4	2026/10/28 11:00:00	架空顧客B	スタンプ(446/1988)		受信	スタンプ	Udemo0002	m003	EV003
5	2026/10/28 10:05:00	架空顧客A	ご連絡ありがとうございます。担当よりお見積書をお送りします		送信	テキスト	Udemo0001		
6	2026/10/28 10:15:00	架空顧客A	お見積書です。ご確認ください		送信	テキスト	Udemo0001		
7	2026/10/28 10:15:00	架空顧客A		https://example.com/demo/estimate.jpg	送信	画像	Udemo0001		
8	2026/10/28 11:05:00	架空顧客B	スタンプありがとうございます		送信	テキスト	Udemo0002		

送信分の取り方メモ（見本）

APIで取れる範囲

・Webhookで届くのは、利用者からのメッセージ（テキスト・画像・スタンプなど）と、友だち追加などのイベントです。
・公式アカウント側から送った分は、Webhookでは届きません。
・そのため、送信分は「送るときに記録する」形にします。GASからMessaging APIで送る返信・プッシュは、送る関数（見本のpushAndLog / replyAndLog）を通せば、送った内容と相手IDを同じシートに書けます。
・受信画像は、メッセージIDで画像を取得するAPIからドライブに保存できます。保存できる期間に限りがあるため、受け取ったときにすぐ保存します。

確認が要る点

・LINE Official Account Managerのチャット画面から手で送った分は、APIの外にあるため、この方法では記録できません。いまは担当者が管理画面から手で送っている運用を想定し、送る場所をシートか送信画面に移す形を主案にします。

見本の主案: 送信もGAS経由にする

・担当者は、シートの「LINE送信」メニューか送信画面から、表示名で相手を選んで送ります。送信と記録が1回で済みます（手順は 使い方.md）。

管理画面からの送信を残す場合の代わりの方法（案）

・主案と併用し、管理画面は急ぎのときだけにする。
・管理画面からの送信は残し、手でバックアップを取る運用にする。
・送信の担当者が、送った内容を簡単なフォームに残す。
どれが合うかは、送信の頻度と担当者の人数を伺ってから決めます。

使い方の手引き（見本）

最初に知っておくこと

・LINE Official Account Managerのチャット画面から手で送った分は、Messaging APIの外にあるため自動では記録できません。
・そこで、担当者が送るときは「シートのメニュー」か「送信画面」から送ります。送ると同時に「トーク履歴」シートに記録されます。

はじめの設定（1回だけ）

・スプレッドシートの「拡張機能」から「Apps Script」を開き、Code.gs と Dialog.html を貼り付けます。
・「プロジェクトの設定」のスクリプトプロパティに、LINE_CHANNEL_ACCESS_TOKEN、SHEET_ID、IMAGE_FOLDER_ID を入れます。
・Webアプリとしてデプロイし、そのURLをLINE DevelopersのWebhook URLに入れます。
・シートを開き直すと、上のメニューに「LINE送信」が出ます。

シートの並び

・トーク履歴: 受信時刻、ユーザー名、受信メッセージ、画像URLの4列は今のままです。5列目から右に、向き（受信・送信）、種類、相手ID、メッセージID、イベントIDを足します。送った分も同じシートに「送信」として入ります。
・送り先: 表示名と相手IDの対応表です。メッセージを受け取ると自動で足されます。同じ表示名の人が2人いるときは、表示名の欄を書き分けてください（例: 架空顧客A（東京））。
・送信: まとめて送るときに使います。表示名、本文、画像URL、状態の4列です。

送り方1: 送信画面から1件ずつ

・メニュー「LINE送信」から「送信画面を開く」を選びます。
・送り先をプルダウンから表示名で選びます。
・本文を入れます。画像も送るときは画像URL欄に入れます。
・「送る」を押すと送信され、トーク履歴に記録されます。

送り方2: 送信シートからまとめて

・「送信」シートに表示名、本文、画像URLを1行ずつ入れます。状態の欄は空のままにします。
・メニュー「LINE送信」から「送信シートの未送信を送る」を選びます。
・状態の欄に「送信済み（日時）」か「エラー（理由）」が入ります。状態が入った行は次から送りません。

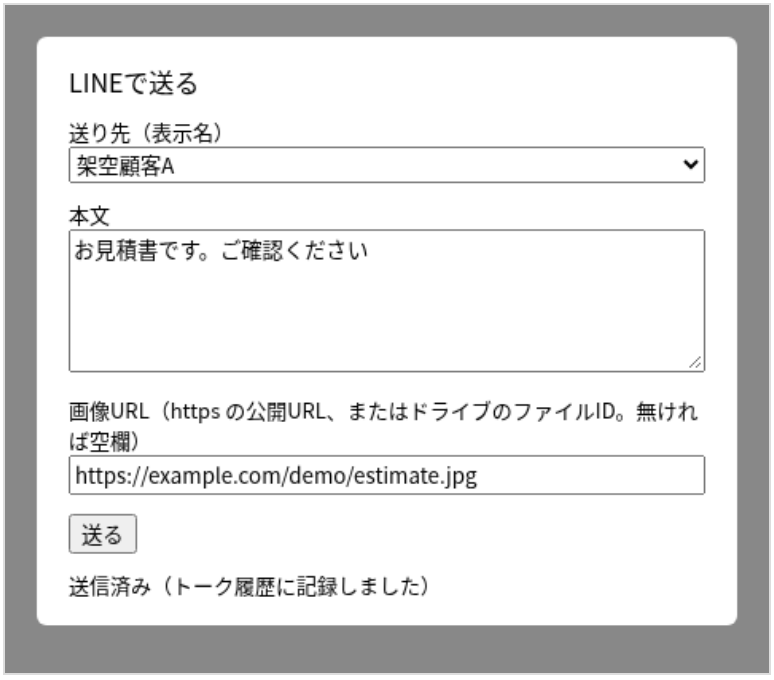
画像を送るとき

・LINEに送れる画像は、https で始まる誰でも開けるURLのものだけです。
・画像URL欄には、そのURLを入れるか、ドライブに置いた画像のファイルIDを入れます。ファイルIDのときは、表示用のURLに直して送ります。
・ドライブの画像は「リンクを知っている全員が閲覧可」にする必要があります。どのフォルダをどこまで共有するかは、ご相談のうえ決めます。

うまくいかないとき

・「送り先に無い表示名です」: 送り先シートに、その表示名の行があるか確かめてください。
・「送信失敗(数字)」: トークンの期限や、相手がブロックしていないかを確かめてください。
・受信が記録されない: Webhook URLとデプロイの版を確かめてください。

※この見本は架空データで、ローカルの模擬環境のテストで確かめたものです。実際のGoogle・LINEでの確認は、契約後に導入先の環境で行います。
・画像の表示は、本番の前にLINEの実機で確かめてください（DriveのURLは途中で別の場所に飛ばされ、表示されないことがあります）。



コードの要点（見本）

1. 表示名で選んで送り、同時に記録

```
function findUserId_(name) {
  var hit = readContacts_().filter(function (c) { return c.name === name; });
  if (hit.length === 0) throw new Error('送り先に無い表示名です: ' + name);
  if (hit.length > 1) throw new Error('同じ表示名が' + hit.length + '人います。送り先シートで名前を分けてください: ' +
name);
  return hit[0].userId;
}

function sendByName(name, text, image) {
  var uid = findUserId_(name);
  var code = pushAndLog(uid, buildMessages_(text, image));
  return code === 200 ? '送信済み' : '送信失敗(' + code + ')';
}

function buildMessages_(text, image) {
  var msgs = [];
  if (text) msgs.push({ type: 'text', text: String(text) });
  if (image) {
    var url = /^https:\/\//.test(image) ? image : driveImageUrl_(image);
    msgs.push({ type: 'image', originalContentUrl: url, previewImageUrl: url });
  }
  if (msgs.length === 0) throw new Error('本文か画像URLのどちらかを入れてください');
  return msgs;
}
```

2. 受信画像はドライブに保存してリンクを記録

```
function saveImage_(messageId, timestampMs) {
  var res = UrlFetchApp.fetch('https://api-data.line.me/v2/bot/message/' + messageId + '/content', {
    headers: { Authorization: 'Bearer ' + getProp_(CONFIG.PROP_TOKEN) },
    muteHttpExceptions: true
  });
  if (res.getResponseCode() !== 200) return '取得失敗(' + res.getResponseCode() + ')';
  var stamp = Utilities.formatDate(new Date(timestampMs), Session.getScriptTimeZone(),
'yyyyMMdd_HHmmss');
  var blob = res.getBlob().setName(stamp + '_' + messageId + '.jpg');
  var file = DriveApp.getFolderById(getProp_(CONFIG.PROP_FOLDER_ID)).createFile(blob);
  return file.getUrl();
}
```

3. 既存の4列は変えず、足す列は右に

```
function getSheet_() {
  var sh = getNamedSheet_(CONFIG.SHEET_NAME, CONFIG.BASE_HEADER.concat(CONFIG.EXTRA_HEADER));
  var base = CONFIG.BASE_HEADER.length;
  if (sh.getLastColumn() < base + CONFIG.EXTRA_HEADER.length) {
    sh.getRange(1, base + 1, 1, CONFIG.EXTRA_HEADER.length).setValues([CONFIG.EXTRA_HEADER]);
  }
  return sh;
}
```

データはすべて架空です。ローカルの模擬環境でテスト22件を通しています。実際のGoogle・LINEでは確認していません。